

Reactive and Safety-Aware Path Replanning for Collaborative Applications

Cesare Tonola¹, Marco Faroni², Saeed Abdolshah³, Mazin Hamad³,
Sami Haddadin^{3,4}, Nicola Pedrocchi¹, Manuel Beschi^{1,5}

Abstract— This paper addresses motion replanning in human-robot collaborative scenarios, emphasizing reactivity and safety-compliant efficiency. While existing human-aware motion planners are effective in structured environments, they often struggle with unpredictable human behavior, leading to safety measures that limit robot performance and throughput. In this study, we combine reactive path replanning and a safety-aware cost function, allowing the robot to adjust its path to changes in the human state. This solution reduces the execution time and the need for trajectory slowdowns without sacrificing safety. Simulations and real-world experiments show the method’s effectiveness compared to standard human-robot cooperation approaches, with efficiency enhancements of up to 60%.

Note to Practitioners—The ISO/TS 15066 outlines collaborative operations, such as Speed and Separation Monitoring (SSM), designed to uphold worker safety in human-robot collaboration. While these safety measures lead to robot deceleration to prevent collisions, they can also cause inefficiencies due to robot inactivity. This work introduces a dynamic real-time adaptation of the robot’s path, aiming to minimize the execution time while considering safety interventions. In contrast to methods relying on pre-computed trajectories, this approach modifies the robot’s path in real-time based on the continuous monitoring of the human state, thereby decreasing safety interventions and downtime, especially in dynamic environments.

Index Terms—Motion re-planning, human-aware motion planning, human-robot collaboration.

I. INTRODUCTION

THE factory of the future envisions seamless collaboration between humans and robots as a means to enhance industrial productivity and improve workers’ conditions [1]. Collaborative applications are subject to safety requirements to mitigate potential hazards to the workers’

health [2]. The ISO/TS 15066 technical specification [3] defines the *Power and Force Limiting* (PFL) and the *Speed and Separation Monitoring* (SSM) modes, which describe safety requirements for applications with and without contacts, respectively. Typically, their implementation restricts the robot’s speed and halts it when close to a human.

Traditional motion planners are poorly suited to collaborative settings. They treat humans as obstacles and do not dynamically adapt to human movements. They generally optimize a cost function, such as minimizing path length or travel time, without considering how safety constraints or human motion may affect the *a-posteriori* execution time. Recent research has tackled this issue at different levels, such as task-planning level [4], [5], control level [6], [7], and motion planning level [8], [9].

Previous studies, such as [9]–[15], exemplify *reactive* planning approaches that adapt a robot trajectory in real-time based on human proximity. Works like [9]–[12] modify only the speed and are effective for brief, infrequent intrusions, but they become inefficient with frequent close collaboration, resulting in repeated safety stops when the human is nearby. In contrast, [13]–[15] dynamically adjust the robot path and prioritize computational efficiency by searching for feasible, minimum-length solutions but do not consider the impact of safety requirements on the path.

Another class of approaches involves *proactive* planners [16]–[18]. They pre-compute trajectories just before the motion starts by optimizing *ad-hoc* cost functions beyond minimizing path length. For example, they reduce expected interference between the operator and the robot, relying on human behavior models to minimize collision risk [16], execution time [17], or human discomfort [18]. However, the computational demands of these cost functions generally make them unsuitable for real-time path adjustments. As a result, proactive planners lack online adaptability, and their effectiveness diminishes in settings where the operator’s movements are unpredictable.

In such a context, our goal is to get the best from both reactive and proactive approaches so that we can replan the robot path online according to the human state with a rate compatible with runtime implementation (≥ 5 Hz) and minimize the trajectory traversal time during execution. Our approach uses a safety-aware cost function based on ISO/TS 15066 that continuously estimates the

¹Institute of Intelligent Industrial Technologies and Systems, National Research Council of Italy, STIIMA-CNR, Milan, Italy.
{cesare.tonola}@stiima.cnr.it

²Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milan, Italy.

³Munich Institute of Robotics and Machine Intelligence (MIRMI), Technical University Munich, Munich, Germany.

⁴Mohamed Bin Zayed University of Artificial Intelligence, Masdar City, Abu Dhabi, UAE.

Munich Institute of Robotics and Machine Intelligence (MIRMI), Technical University Munich, Munich, Germany.

⁵Dipartimento di Ingegneria Meccanica e Industriale, University of Brescia, Italy.

expected trajectory execution time—factoring in potential safety stops and slow-downs—and integrates it into a real-time path replanning algorithm. To achieve real-time performance, we speed up the trajectory re-computation by (i) restricting the search space through a search heuristic and (ii) exploiting prior search information and employing lazy cost function evaluations. This balances computational complexity with speed, significantly boosting the efficiency of online path search. The proposed method is called “Human-Aware Multi-pAth Replanning Strategy” (MARSHA), and it allows for continuous adaptation to unexpected human movements while still providing safety-aware solutions. It dynamically estimates the expected execution time of the robot’s movements and minimizes it by adjusting the trajectory. We evaluated our method with simulations and real experiments (video available at [19]). A ROS [20] compatible algorithm implementation is available at [21].

II. RELATED WORKS

Human-aware motion planners enable robots to move while considering the human presence. Previous works focused on optimizing paths by considering the human-robot distance, the human field of view, and comfort [18], [22]. Other approaches avoid high-occupancy areas by acting on the planner cost function [23]–[25]. For example, [23] biases the sampling to regions situated far from the current human state [24], [25] utilized STOMP [26] to minimize a cost function penalizing previously occupied regions while maximizing the human-robot distance, and [16] deformed the trajectory based on a repulsion field associated with checkpoints on the human skeleton. [27] accounts for collision risk, social norms, and crowded areas. [28] focused on reducing execution time and the risk of robot–worker collisions in handover tasks. Some studies evaluated human factors and demonstrated enhancements in work fluency and operator satisfaction when a human-aware motion planner is adopted [29], [30]. Recent works dealt with the safety integration within the motion planner. [17] and [31] evaluate paths by estimating the speed scaling when the robot crosses those paths. [32] uses spatiotemporal human occupancy maps to find robot trajectories that anticipate human movements. These methods consider speed limitations in motion generation. They do not require tuning and minimize an estimate of the execution time according to SSM [17], [32] and PFL [31] rules. However, these methods lack robustness to human behavior variations and real-time adaptability.

Path replanning algorithms allow for quick adjustments to the robot’s current trajectory, ensuring fast reactions to environmental changes, whereas recalculating a path from scratch would be time-consuming and reduce the robot’s overall reactivity. Four main approaches can be identified for algorithms dealing with real-time path modification:

1) *Graph-based methods*: They build a graph on the discretized search space and modify the path when new obstacle-related information emerges [33], [34]. Their suitability is limited to small-dimensional problems like mobile robot navigation due to poor scalability.

2) *Potential fields methods*: These methods modify precomputed trajectories using virtual attracting and repelling forces generated by obstacles and goals [35]–[37]. Despite the high reactivity, the robot may get stuck in local minima, and it does not guarantee finding a valid solution.

3) *Learning-based methods*: These methods deliver reactive and efficient performance in environments similar to their training conditions. Most approaches compute the robot’s next action (e.g., position or velocity command) based on the current state of the robot and environment, using techniques such as Reinforcement Learning or Graph Neural Networks [38]–[41]. However, they often struggle to generalize effectively to new environments.

4) *Sampling-based methods*: These algorithms, like RRT and PRM variants [14], [15], [42]–[44], are commonly used for path planning without upfront discretization. They are popular thanks to their real-time adaptability, scalability, and robustness to dynamic environments and complex scenarios. Recently, a sampling-based Multi-pAth Replanning Strategy (MARS) [14] has been proposed by the authors to address replanning in complex search spaces. MARS uses a set of precomputed paths to find a solution and refine it over time rapidly. The algorithm reuses existing subtrees and precomputed paths during the optimization and it exploits a parallel architecture. However, it focuses on finding the shortest path, without embedding any effects the safety module may have on the path itself, leading to further slowdowns.

III. PRELIMINARIES

A. Path Planning Formulation

Path planning involves determining a path σ (i.e., a sequence of feasible robot’s configurations) that connects an initial configuration $q_{\text{start}} \in \mathcal{C}$ to a final configuration $q_{\text{goal}} \in \mathcal{C}$. Here, $\mathcal{C} \subseteq \mathbb{R}^n$ is the robot’s configuration space, n is the number of degrees of freedom of the robot, and q is the vector of robot’s joint angles. A configuration q is considered feasible if it belongs to the collision-free configuration set $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$. A cost function assigns a non-negative scalar cost c to the path. The optimal path $\sigma^* : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ is the one that minimizes the cost function:

$$\sigma^* = \arg \min_{\sigma} c(\sigma) \quad \text{s.t.} \quad \sigma(0) = q_{\text{start}}, \sigma(1) = q_{\text{goal}}, \quad (1)$$

A common choice for c is the path length. For example, if σ is discretized into w waypoints, we have:

$$c(\sigma) = \sum_{i=1}^{w-1} \|q_{i+1} - q_i\|_2 \quad (2)$$

In the context of safety-aware path planning, [17] evaluates the cost as the estimated path execution time t_{est} which embeds safety constraints based on the human state $\mathcal{H}$:

$$c(\sigma) = t_{\text{est}}(\sigma, \mathcal{H}) \quad (3)$$

By discretizing σ , (3) can be expressed as follows:

$$c(\sigma) = \sum_{i=1}^{w-1} t_{\text{nom},i} \lambda_i(q_i, q_{i+1}, \mathcal{H}) \quad (4)$$

where $t_{\text{nom},i}$ is the nominal execution time (no human in the scene), and λ_i is the average time dilation factor (see Section IV-A) that measures the effect of the human on the execution time of connection $\overline{q_i q_{i+1}}$, i.e.:

$$\lambda_i = \frac{t_{\text{est},i}(\mathcal{H})}{t_{\text{nom},i}} \geq 1 \quad (5)$$

Note that $t_{\text{nom},i}$ is usually obtained *a posteriori* using a path parametrization method (e.g., TOPP [45]) as each segment's timing is influenced by the kinematic constraints of adjacent segments. It can be underestimated by considering the minimum time required to cross the connection $\overline{q_i q_{i+1}}$, imposing that at least one joint is moving at its maximum speed. This leads to an overestimation of the velocity in favor of safety. The minimum time required to travel $\overline{q_i q_{i+1}}$ is therefore given by the maximum of the component-wise ratio $(q_{i+1} - q_i) \oslash \dot{q}_{\text{max}}$, where $\dot{q}_{\text{max}}$ denotes the maximum joint speed vector; summarizing, (4) becomes:

$$c(\sigma) = \sum_{i=1}^{w-1} \left\| (q_{i+1} - q_i) \oslash \dot{q}_{\text{max}} \right\|_{\infty} \lambda_i(q_i, q_{i+1}, \mathcal{H}) \quad (6)$$

B. Safety Requirements

Safety requirements for collaborative applications are defined by ISO/TS 15066 (*Robots and robotic devices – Collaborative robots* [3]). The document outlines various collaborative operations, with two of particular interest.

1) *Speed and Separation Monitoring (SSM)*: focuses on maintaining a minimum distance between the human and the robot to ensure the robot can stop before possible impacts. Specifically, the minimum human-robot distance, denoted as S , should always satisfy the condition $S \geq S_p$. Here, S_p represents the protective separation distance. We can derive the maximum safe velocity v_{max} of the robot towards the human based on the current separation distance S :

$$v_{\text{max}} = \sqrt{v_h^2 + (a_s T_r)^2 - 2a_s(C - S) - a_s T_r - v_h} \quad (7)$$

where v_h is the human velocity towards the robot, a_s is the maximum Cartesian deceleration of the robot towards the human, T_r is the reaction time of the system, and C is a parameter accounting for the uncertainty of the perception system.

2) *Power and Force Limiting (PFL)*: allows the robot to come into contact with a human with non-zero speed, as long as the kinetic energy transferred to the human does not exceed a threshold E_{max} . E_{max} is turned into a limit on the maximum robot speed:

$$v_{\text{max}} = \frac{F_{\text{max}}}{\sqrt{k}} \sqrt{m_r^{-1} + m_h^{-1}} \quad (8)$$

where F_{max} and k are the maximum contact force and spring constant for a specific body region and m_r and m_h are the effective mass of the robot and of the human body region, respectively. See [3], [46] for more information.

Both SSM and PFL constrain the robot's maximum speed. For brevity, we will focus exclusively on SSM in the experimental campaign described in Sections V and VI, without compromising the generality of our methodology.

IV. PROPOSED APPROACH

The proposed approach operates within the framework of Fig. 1. This paradigm consists of an *offline path planner* responsible for calculating the initial path and a *fast safety-aware replanner*, online updating the trajectory based on information about the human state. Additionally, the offline planner can proactively compute a trajectory that reduces the activation of safety rules. A safety module implements ISO/TS 15066 to ensure safe cooperation. The replanner modifies the path aiming to minimize the intervention required by the safety module, thereby enhancing the overall efficiency of cooperation. This goal is achieved by minimizing a safety-aware cost function derived from (6) and providing an *admissible informed set* for the cost function. Including an admissible informed set in a sampling-based path planner speeds up the convergence rate by discarding areas that do not contain the optimal solution, accelerating the online replanning process.

A. The Safety-Aware Cost Function

The main objective of this cost function is to minimize the execution time while considering the human state. Simultaneously, in Section IV-B, we define an admissible informed set that can be directly sampled to accelerate the replanning process. The proposed cost function is:

$$c(\sigma) = \sum_{i=1}^{w-1} \left\| (q_{i+1} - q_i) \oslash \dot{q}_{\text{max}} \right\|_2 \lambda_i(q_i, q_{i+1}, \mathcal{H}) \quad (9)$$

The cost function calculates the length of the path segments, weighting each joint by its maximum speed. Additionally, it incorporates the penalty term λ to discourage paths that reduce collaboration efficiency. It is worth noting that (9) serves as an upper limit compared to (6).

In fact, (9) uses the $\mathcal{L}_2$ norm instead of the $\mathcal{L}_\infty$ norm and $\|v\|_2 \geq \|v\|_\infty, \forall v \in \mathbb{R}^n$. Thus, we have:

$$\sum_{i=1}^{w-1} \|(q_{i+1} - q_i) \odot \dot{q}_{\max}\|_2 \lambda_i(q_i, q_{i+1}, \mathcal{H}) \geq \sum_{i=1}^{w-1} \|(q_{i+1} - q_i) \odot \dot{q}_{\max}\|_\infty \lambda_i(q_i, q_{i+1}, \mathcal{H}) \quad (10)$$

Therefore, by minimizing the proposed cost function, we decrease the upper bound of (6) and consequently reduce the estimated execution time of the path.

The safety guidelines outlined in ISO/TS 15066 can be translated into a human-robot relative speed limit, imposing that the robot's velocity towards the human must not exceed the maximum allowed robot speed v_{max} according to Section III-B. If the nominal robot velocity towards the human v_{rh} is greater than v_{max} , the robot will be slowed down by a factor $v_{rh}/v_{max} > 1$, resulting in an expected time dilation factor $\lambda > 1$. We can compute v_{rh} and λ for a generic configuration q according to [17] as follows. We consider a set of m points of interest for the human state $\mathcal{H} = \{h_1, h_2, \dots, h_m\}$ and a set of p points of interest on the robot's structure $\mathcal{R} = \{r_1, r_2, \dots, r_p\}$. Each tuple $\langle r_j, h_k \rangle \in \mathbb{R}^3 \times \mathbb{R}^3$ denotes the Cartesian positions of a pair of key points, with one point on the robot structure and the other on the human skeleton. Each point r_j is associated with the robot's configuration q through the forward kinematic $r_j = \text{fk}_j(q)$. Moreover, $u_{\langle r_j, h_k \rangle}$ is the unit vector from r_j to h_k :

$$u_{\langle r_j, h_k \rangle} = \frac{h_k - r_j}{\|h_k - r_j\|_2} \quad (11)$$

and $\dot{r}_j, \dot{h}_k$ are the velocities of r_j and h_k , respectively. The scalar velocity $v_{\langle r_j, h_k \rangle}$ of the robot point r_j towards the human point h_k is computed as:

$$v_{\langle r_j, h_k \rangle} = (\dot{r}_j - \dot{h}_k) \cdot u_{\langle r_j, h_k \rangle} = (J_j(q)\dot{q} - \dot{h}_k) \cdot u_{\langle r_j, h_k \rangle} \quad (12)$$

where the operator $\cdot$ denotes the scalar (dot) product between two vectors and $J_j(q)$ denotes the Jacobian of r_j . A positive value of $v_{\langle r_j, h_k \rangle}$ is used when the distance between r_j and h_k is decreasing.¹ As stated in Section III, robot velocity is overestimated in favor of safety by considering that at least one joint is moving at its maximum speed. Hence, for connection $\overline{q_i q_{i+1}}$ we have:

$$u_i = \frac{(q_{i+1} - q_i)}{\|q_{i+1} - q_i\|_2}, \quad \dot{q}_i = \left(\min_{l=1, \dots, n} \left| \frac{\dot{q}_{\max, l}}{u_{i, l}} \right| \right) u_i \quad (13)$$

where n is the robot's degrees of freedom, and l is the joint index. In (5) and (9), λ_i represents the estimated time dilation factor experienced by the robot while traversing the connection $\overline{q_i q_{i+1}}$. However, since we are assuming

¹The sign of $v_{\langle r_j, h_k \rangle}$ depends on the direction of $u_{\langle r_j, h_k \rangle}$ (from robot to human) and the velocity $(\dot{r}_j - \dot{h}_k)$ (from human to robot).

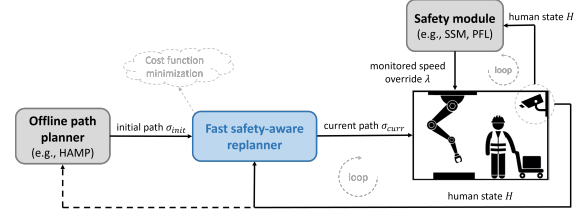


Fig. 1: System overview.

a constant robot velocity along the connection, it can also be computed as the ratio between velocities v_{rh} and v_{max} . Thus, the time dilation factor $\lambda(q)$ at a given robot configuration q can be calculated as:

$$\lambda(q) = \max_{\substack{j=1, \dots, p \\ k=1, \dots, m}} \left(\frac{v_{\langle r_j, h_k \rangle}}{v_{max}}, 1 \right) \geq 1 \quad (14)$$

where both $v_{\langle r_j, h_k \rangle}$ and v_{max} are scalar quantities. Equation (14) computes $\lambda(q)$ for a single configuration q . To determine λ_i for the entire connection, we divide $\overline{q_i q_{i+1}}$ into z equally spaced configurations q_ζ , with $\zeta = 1, \dots, z$, compute the slowdown factor for each configuration $\lambda_\zeta = \lambda(q_\zeta)$, and finally calculate the average slowdown as $\lambda_i = \frac{1}{z} \sum_{\zeta=1}^z \lambda_\zeta$. The slowdown computation can be parallelized for efficiency. By substituting (7) or (8) in (14) makes the cost function (9) penalizing the paths that fail to meet the SSM or PFL requirements.

B. An Admissible Informed Set for the Cost Function

Given a positive cost function $c(\cdot)$, the cost of the optimal path from q_{start} to q_{goal} passing through a generic point $q \in \mathcal{C}_{\text{free}}$ is equal to the optimal cost-to-come (from q_{start} to q), plus the optimal cost-to-go (from q to q_{goal}). If a heuristic function that never overestimates the cost function is available, it is possible to discard regions of the sampling space that will never contain the optimal path, thus speeding up the planning process. Such function is called an *admissible heuristic* and the remaining sampling space is known as an *admissible informed set*.

For instance, if $c(\cdot)$ is the path length, the Euclidean norm is an admissible heuristic because:

$$\|q - q_{\text{start}}\|_2 + \|q_{\text{goal}} - q\|_2 \leq \min(c(q_{\text{start}}, q) + c(q, q_{\text{goal}})) \quad (15)$$

where the right-hand side represents the cost of the optimal path from q_{start} to q_{goal} passing through q .

Let $c_{\text{best}, i}$ be the cost of best solution found up to iteration i of the planner. Because $\min(c(q_{\text{start}}, q) + c(q, q_{\text{goal}})) \leq c_{\text{best}, i}$, it follows that:

$$\|q - q_{\text{start}}\|_2 + \|q_{\text{goal}} - q\|_2 \leq c_{\text{best}, i} \quad (16)$$

Suppose (16) does not hold for a point q . Then, the best possible path through q has a higher cost than $c_{\text{best}, i}$, meaning that q cannot belong to the optimal path. In general, all points outside the admissible informed set

$\mathcal{I} = \{q \in \mathcal{C}_{\text{free}} \mid \|q - q_{\text{start}}\|_2 + \|q_{\text{goal}} - q\|_2 < c_{\text{best},i}\}$ are guaranteed not to improve $c_{\text{best},i}$. By randomly sampling directly from $\mathcal{I}$, rather than from the entire configuration space, we speed up the planner convergence rate [47].

Considering that $\lambda \geq 1$ in (9), it is possible to define an admissible heuristic for (9) as:

$$\tilde{c}(q) = \|(q - q_{\text{start}}) \odot \dot{q}_{\text{max}}\|_2 + \|(q_{\text{goal}} - q) \odot \dot{q}_{\text{max}}\|_2 \quad (17)$$

from which the following admissible informed set derives:

$$\tilde{\mathcal{I}} = \{q \in \mathcal{C}_{\text{free}} \mid \|(q - q_{\text{start}}) \odot \dot{q}_{\text{max}}\|_2 + \|(q_{\text{goal}} - q) \odot \dot{q}_{\text{max}}\|_2 < c_{\text{best},i}\} \quad (18)$$

Note that (17) differs from (15) because (17) weighs each joint by the inverse of its maximum speed. If we define $\hat{q} = \hat{g}(q) = q \odot \dot{q}_{\text{max}}$ and $\hat{\mathcal{C}}_{\text{free}} = \hat{g}(\mathcal{C}_{\text{free}})$, (18) becomes:

$$\hat{\mathcal{I}} = \{\hat{q} \in \hat{\mathcal{C}}_{\text{free}} \mid \|\hat{q} - \hat{q}_{\text{start}}\|_2 + \|\hat{q}_{\text{goal}} - \hat{q}\|_2 < c_{\text{best},i}\} \quad (19)$$

Hence, it is possible to sample from the informed set (18) by directly sampling from (19) (see [47] for efficient sampling algorithms). The sampled point $\hat{q}$ is then mapped back to the original space using the inverse transformation $q = \hat{g}^{-1}(\hat{q})$, yielding a valid sample q from (18).

C. The Path Replanning Algorithm

In this section, we present the Human-Aware Multi-path Replanning Strategy (MARSHA), which extends MARS [14] to handle the new cost function while maintaining responsiveness and replanning capabilities.

MARS is a sampling-based anytime path replanning algorithm that operates on a set $\mathcal{S}$ of pre-calculated paths connecting the same start-goal pair. MARS connects the current path to nodes on the available paths rather than directly connecting to the goal and uses the available sub-path from those nodes toward the goal. Once an initial solution is obtained, MARS employs heuristics and informed sampling to refine the solution over time iteratively. The algorithm reuses valid subtrees in connecting the current path with the available nodes to reduce computational overhead. It is worth noting that MARS has been proven to be *probabilistic complete* and *asymptotic optimal* when used in conjunction with an asymptotically optimal planner for subtree growth (see [14] for details).

The algorithm uses three parallel threads to maximize performance. The *Trajectory execution* thread interpolates the trajectory at a high rate and sends corresponding commands to the robot's controller. The *Path replanning* thread handles the replanning process, continually looking for new paths, refining the solution, and computing a trajectory based on it. The *Collision check* thread checks the collisions along the current path σ_{curr} and the available paths $\mathcal{S}$. Algorithms 1, 2, and 3 show the algorithmic changes to obtain MARSHA from MARS. We adopted a lazy approach that minimizes the need for frequent evaluations of the cost function (line 10 of Algorithm 2) to tackle

the computational load caused by the cost function, which is significantly more demanding than the simple Euclidean norm used in MARS. During the subtree expansion, our algorithm employs a strategy that only evaluates the cost of new connections when necessary. If the selected path planning algorithm does not use a rewiring procedure (e.g., RRT [48]), the cost of connections is assessed after the subtree has successfully connected to its target node (lines 5 and 8 of Algorithm 3), limiting the cost function calculation to the connections that belong to the solution.

The evaluation of the cost function requires the updated human state, $\mathcal{H}$ (line 17 of Algorithm 1). For example, a human tracking system can provide $\mathcal{H}$. The collision check thread also updates $\mathcal{H}$ (line 6) and calculates the cost of the connections for the paths accordingly (line 12). This approach ensures that MARSHA does not waste time evaluating the cost of connections to the current path but focuses on the connections belonging to the new solutions that have not yet been evaluated in the iteration. For further details on sub-algorithms that remain unchanged from MARS, readers may refer to [14].

V. SCENARIO ANALYSIS

This Section summarizes the effectiveness of the methods covered in Section II, including sampling-based path planners [49]–[51], human-aware [16], [18], [23] and safety-aware [17], [31], [32] proactive planners, reactive path replanners [14], [15], [42], [43], and learning-based approaches [38]–[41]. First, in Section V-A, we classify the methods according to their capabilities and describe a paradigmatic application to compare them. Then, in Section V-B, we assess the methods considering different interaction scenarios for the target application.

A. Targeted Application

To evaluate the performance of the aforementioned methodologies, we consider the planner's capabilities:

Reactivity: The ability to dynamically adjust the robot's path online based on the human's movements.

Human-aware proactivity: The ability to maintain human-robot distance or avoid areas typically occupied by humans.

Safety-aware proactivity: The ability to compute trajectories in line with ISO/TS 15066 safety guidelines.

Robustness to changing working conditions: The extent to which a planning pipeline adapts to new tasks or environments, indicating how challenging it is to apply the planner across different scenarios.

Table I reports the classifications of the aforementioned methodologies according to such capabilities. As a targeted application, we consider a cobot that palletizes boxes, and a human operator can access the workspace to inspect or move pallets. The robot can move flexibly

Algorithm 1 The threads that operate in parallel to run and support MARSHA

1: Thread <i>Trajectory execution</i>	5: Thread <i>Collision check</i>	15: Thread <i>Path replanning</i>
2: $t = t + \Delta t$	6: $\mathcal{H} \leftarrow \text{updateHumanState}()$	16: $q_{\text{curr}} \leftarrow \text{projectOnPath}(state, \sigma_{\text{curr}})$
3: $state \leftarrow \text{sampleTrajectory}(\tau, t)$	7: $\sigma_{\text{subpath}} \leftarrow \sigma_{\text{curr}}[q_{\text{curr}}, q_{\text{goal}}]$	17: $\sigma_{\text{RP}}, solved \leftarrow$
4: $\text{sendToController}(state)$	8: $\mathcal{P} \leftarrow \mathcal{S} \cup \sigma_{\text{subpath}}$	$\text{replan}(\sigma_{\text{curr}}, q_{\text{curr}}, \mathcal{S}, \Delta t_{\text{max}}, \mathcal{H})$
	9: for $\sigma_j \in \mathcal{P}$ do	18: if <i>solved</i> then
	10: $free \leftarrow \text{checkCollision}(\sigma_j)$	19: $\sigma_{\text{curr}} \leftarrow \sigma_{\text{RP}}$
	11: if <i>free</i> then	20: $\tau \leftarrow \text{computeTrajectory}(\sigma_{\text{curr}})$
	12: $c_{\sigma_j} \leftarrow c(\sigma_j, \mathcal{H})$	
	13: else	
	14: $c_{\sigma_j} \leftarrow +\infty$	

	reactivity	human-aware proactivity	safety-aware proactivity	robustness to changing working conditions
Standard planners [49]–[51]				✓
Human-aware planners [16], [18], [23]		✓		✓
Safety-aware planners [17], [31], [32]		✓	✓	✓
Reactive replanners [14], [15], [42], [43]	✓			✓
Learning-based planners [38]–[41]	✓	✓*		
MARSHA	✓	✓	✓	✓

* Only those methods trained with humans in the shared workspace

TABLE I: Capabilities of the proposed approach and other works in the literature.

Algorithm 2 MARSHA: high-level description

```

1: Define  $\mathcal{P}$  as the set of available paths
2: Define  $\mathcal{Q}_1$  as the queue of the nodes of  $\sigma_{\text{curr}}$  between  $q_{\text{curr}}$  and the obstacle
3: Sort  $\mathcal{Q}_1$  by some criterion
4: for each node  $q_n \in \mathcal{Q}_1$  do
5:   Insert the nodes of each  $\sigma \in \mathcal{P}$  into a queue  $\mathcal{Q}_2$ 
6:   Sort  $\mathcal{Q}_2$  by some criterion
7:   for each  $q_j \in \mathcal{Q}_2$  do
8:     Define the informed set on the best cost up to now
9:     Get the subtree rooted in  $q_n$  from the informed set
10:    Grow the subtree in the informed set to reach  $q_j$  with lazy cost evaluation
11:   Update  $\mathcal{Q}_1$  if a solution was found
12: Get the best path from the graph

```

Sub-Algorithm 3 Connect Node to Paths

```

1: function GROWINELLIPSOID( $\mathcal{T}, E_{\text{U}}, q_j, t_{\text{MAX}}$ )
2:   while  $\neg ok$  &  $iter < \text{max\_iter}$  &  $t < t_{\text{MAX}}$  do
3:      $iter = iter + 1$ 
4:      $q \leftarrow \text{sampleInEllipsoid}(E_{\text{U}})$ 
5:      $q_{\text{new}} \leftarrow \mathcal{T}.\text{growTree}(q)$  > no cost function evaluation
6:     if  $\|q_{\text{new}} - q_j\| < \text{min\_dist}$  then
7:       if path from tree root to  $q_j$  is valid then
8:         compute the cost function for the path found
9:         create a second-order connection from  $q_{\text{new}}$  to  $q_j$ 
10:         $ok \leftarrow \text{True}$ 
11:       else
12:         hide the invalid branch temporarily from  $\mathcal{T}$ 
13:   return  $ok$ 

```

between its designated positions, while human interventions vary in duration and frequency, from brief, sporadic entries to a longer presence in the workspace. This setup often involves close human-robot proximity [52]. A safety perception system monitors the human's position and communicates it to a safety module, which slows the robot down in compliance with ISO/TS 15066.

B. Scenarios

1) *Short and sporadic human presence:* The human operator occasionally enters the robot's workspace unpredictably to briefly retrieve objects for inspection, triggering the robot's safety module and causing temporary slowdowns. Referring to Table I, algorithms that lack "reactivity" struggle in these situations. Proactive planning methods depend on predicting the operator's behavior through probabilistic models, which is difficult due to the infrequent entries. In contrast, reactive planning algorithms can modify the robot's path in real-time when the operator is detected. As these human entries are short and infrequent, their impact on the robot's efficiency remains minimal, and simply relying on the safety module is often adequate for maintaining smooth operations.

2) *Short and frequent human presence:* The human operator frequently enters the workspace briefly to feed the station with new boxes or take completed pallets away. These frequent entries often trigger the robot's safety module, causing repeated slowdowns or complete stops, significantly reducing efficiency. As in the case of sporadic entries, proactive planning approaches fail to keep up with the unpredictability of frequent interventions. Reactive planning algorithms can adjust the robot's path but if they lack "safety-awareness" they may still lead to significant safety interventions. Frequent occurrences of these slowdowns can accumulate, negatively impacting productivity. An approach that combines reactive path adjustments with safety-aware strategies is more effective at minimizing disruptions and maintaining overall efficiency.

3) *Prolonged and structured human presence:* The human remains in the workspace for an extended time following predictable patterns, such as the robot placing boxes on a pallet while the operator closes each one.

Deviations in the operator’s movements are unlikely. This predictability enables the robot to plan its motions more effectively around the human’s position. “Human-aware” proactive algorithms factor in the operator’s position to compute paths that maintain a safe distance. For instance, learning-based approaches can learn policies that guide the robot to avoid proximity to humans, thereby minimizing the risk of collisions and reducing the need for frequent safety interventions. Incorporating “safety-aware proactivity” further optimizes the robot’s travel time by reducing delays and ensuring smoother operations.

4) *Prolonged and unstructured human presence*: The operator stays in the workspace for an extended time with unpredictable movements. For instance, the operator may repeatedly move pallets or boxes between different locations within the workspace, with no fixed pattern, perhaps rearranging items to prepare for inspections or transfers. Here, even “safety-aware” algorithms lacking “reactivity” capabilities fail to prevent the robot from long-term potential safety stops when unexpected obstructions occur. These algorithms cannot cope with human movement which significantly deviates from expected patterns.

5) *Changes to task or environment*: The robot’s tasks, workspace layout, or parameters may change frequently in highly dynamic environments, such as those typical in high-mix, low-volume manufacturing. Examples include relocating pick-up areas, accommodating obstacles like accumulated parts, integrating new operators, or adapting to different tasks as production fluctuates. Traditional sampling-based planners adjust well to such conditions, leveraging vision system inputs to respond to new configurations. However, learning-based methods, often pre-trained for specific robots and fixed environments, may require substantial adaptation retraining, which is impractical for rapidly changing setups.

The performance of the various algorithms in the described scenarios is illustrated in Fig. 2. MARSHA promises strong performance across all scenarios due to its combination of proactive, safety-aware planning and reactive capabilities and inherent adaptability to dynamic tasks and changing environments.

VI. EXPERIMENTS

We experimentally compare MARSHA with reactive and proactive approaches, prioritizing those that share MARSHA’s ability to adapt to varying working conditions.

A. Selection of Reactive Approaches Rationale

We evaluated two approaches in comparison to MARSHA. The first approach, dSSM (dynamic SSM 2D [11]), continually monitors both human and robot statuses, determining the maximum allowable robot speed based on (7). If the robot’s speed exceeds the maximum allowed by safety considerations, it is scaled by a factor λ . Concerning Fig. 1, there is no replanning block. The second

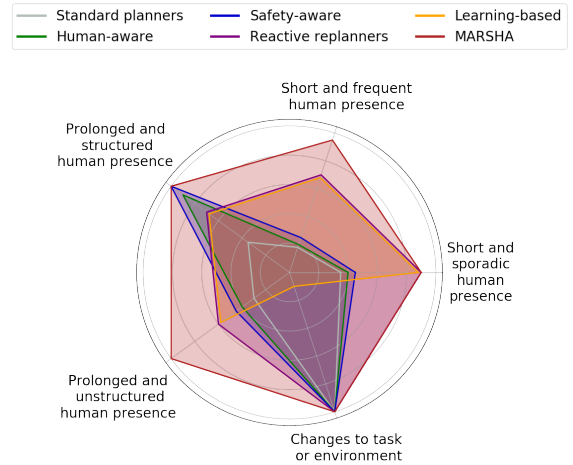


Fig. 2: Comparison of algorithm categories listed in Table I considering the scenarios in Section V. Each scenario assumes a safety module based on ISO/TS 15066’s SSM guidelines. The closer an algorithm’s shape extends to the outer perimeter of the chart, the better it performs.

reactive approach used in the experiments combines dSSM with a path replanning algorithm (MARS [14]). In the first case, the robot slows down according to the safety criterion defined by (7). In the second case, the robot has the additional capability to replan its path. The dSSM approach holds the robot as long as the operator is nearby, while MARS would modify the path to find collision-free solutions but at significantly reduced speed. It is worth noting that MARS modifies the path only when encountering an obstacle or identifying a shorter solution. Instead, MARSHA predicts the effect of the safety module on the solutions found and aims to minimize the traversing time, considering the speed slowdowns due to human movements. MARS and MARSHA were assigned a maximum calculation time of 200 ms².

B. Selection of Proactive Approaches Rationale

For proactive methods, MARSHA was compared to a safety-aware path planning algorithm (HAMP [17]) and a minimum-length path planning algorithm (MIN-LEN), where the human’s expected volume is treated as an obstacle. The MIN-LEN algorithm refers to an optimal path planning algorithm with sufficient computational time to practically achieve the optimal solution, such as RRT* [49], BIT* [50], or other optimal planners (e.g., [51], [53])³. Both approaches were integrated with dSSM during runtime. HAMP aims to find a time-optimal solution by minimizing (6), generating paths that require minimal or no intervention from the safety module as long as the

²As per Algorithm 1, the robot tracks the trajectory at a high rate (e.g., 500 Hz). In contrast, trajectory updates occur at a rate up to 5 Hz.

³These experiments focus only on the time required to *execute* a trajectory in a collaborative scenario, not in the offline computation time.

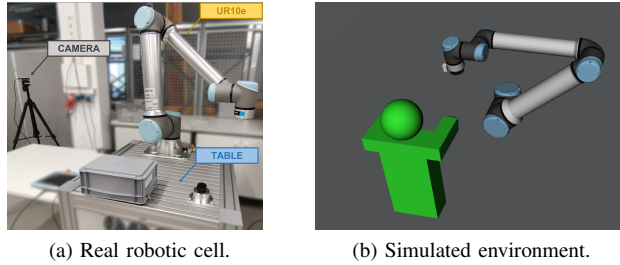


Fig. 3: Experimental setups.

operator remains within the expected space defined during the planning phase. However, HAMP's performance deteriorates if the operator deviates from this space. Conversely, MIN-LEN seeks to identify the shortest path while treating the human as an obstacle. This approach results in shorter paths but at the expense of more frequent safety module activations. Referring to Fig. 1, tests with HAMP and MIN-LEN do not implement replanning blocks.

C. Design of Experiments and Metrics

We evaluate the following metrics: the *Trajectory execution time*, which measures the time taken to complete the task, normalized relative to the minimum execution time (*i.e.*, the time obtained when executing the trajectory without human presence), and the *Average scaling factor* (ranging from 0 to 100), which represents the intervention of the safety module. A lower average scaling factor indicates a higher level of intervention by the safety module in reducing the robot's speed.

We conduct simulated and real-world experiments. The setup (Fig. 3a) comprises a Universal Robot UR10e, a shared table, and an Intel RealSense D435. We compute the human skeleton bounding boxes and feed them into the ISO/TS 15066 computation. In simulations, we replicate the human movements using a mannequin (Fig. 3b) with added uniform noise (max. amplitude of 3 cm). The locations of the dummy's head, torso, arms, and hands are used for the safety computation. During real experiments, we limit the robot's speed to 30% of its maximum for safety reasons. SSM implements (7) with $T_r = 0.15$ s, $a_s = 2.5$ m/s², $C = 0.25$ m and $v_h = 1.6$ m/s.

The control pipeline follows Fig. 1. MARSHA and the reactive methods are implemented in the "fast safety-aware replanner" module, while the proactive methods are in the "offline path planner" module. The code is in C++ and executed in ROS [20]. The replanning algorithm were implemented in OpenMORE [54], an open-source library for robot motion replanning.

The tasks consist of a workspace-sharing application. The robot's goal is to move from an initial configuration to a designated target configuration to execute tasks like pick&place operations. Concurrently, an operator approaches and works on the same table.

D. Comparison with Reactive Approaches

1) *Simulation*: The experiments primarily varied based on the duration of the operator's presence at the workbench. The tests were carried out as follows. The robot plans a path from start to goal while the operator is still far away. The trajectory initially corresponds to the fastest one in the absence of the operator. Two additional paths are computed for MARS and MARSHA, as discussed in Section IV-C. As the robot moves, the operator approaches it to work on the shared table. Depending on the algorithm used, the robot slows down or replans its path. After a pre-established time interval, the operator leaves.

We consider three cases: *short*, *medium*, and *long*, which are distinguished by the time during which the operator is close to the robot, which is 5, 10 and 20 seconds, respectively. 50 simulations and 20 experiments on the real cell were performed for each type of test and algorithm. Fig 4 shows snapshots of MARSHA and dSSM running during the experiments. Fig. 5 depicts the outcomes of the *short*, *medium*, and *long* tests conducted in the simulation. A lower trajectory execution time and a higher average scaling factor indicate superior algorithm performance. Upon analysis, it becomes evident that MARSHA outperforms both MARS and dSSM in all tested scenarios, reducing the trajectory execution time by approximately 60% in *long* tests. MARSHA can generate path plans that reduce activations of the safety module, enabling the robot to reach its goal more expeditiously. This behavior remains consistent across all three test variations, regardless of the operator's duration of interaction with the workbench. Conversely, MARS and dSSM exhibit poor performance when the operator remains near the robot for an extended period. dSSM restricts the robot's movement for the entire duration of the operator's presence nearby. MARS's replanning feature enhances safety by ensuring collision-free paths but compromises the achieved performance. Since MARS continuously finds and optimizes new paths in terms of length, the solutions frequently bring the robot near the operator, eliciting strong responses from the safety module. Both MARS and dSSM result in low average scaling factors, indicating that the robot remains mostly stationary during the tests. In contrast, MARSHA exhibits a higher average scaling factor, reflecting fewer safety module interventions. The higher variability of MARSHA scaling factor arises from the stochastic nature of the sampling-based approach and test conditions (*e.g.*, different timing of the human movements). The performance gap between MARSHA and MARS/dSSM is expected to widen as the operator's time at the robot increases. Code to run a simulation is available at [21].

2) *Real scenario*: The findings from the simulations are further validated through experiments conducted on the physical robot, as in Fig. 6. The results show that MARSHA can quickly identify executable paths by considering the impact of the safety module on the robot during

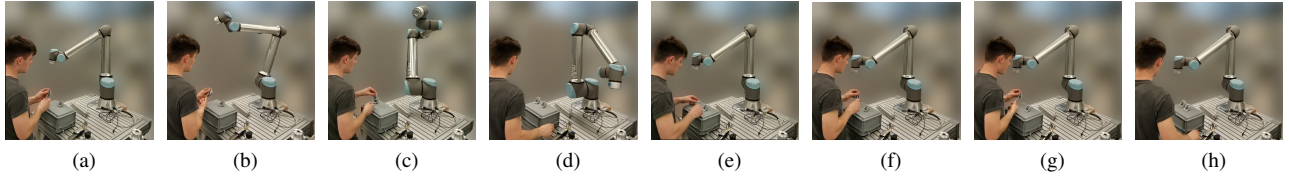


Fig. 4: (a)-(d) MARSHA enables the robot to dynamically adjust its path in response to the operator's approach. (e)-(h) dSSM halts the robot as long as the operator is nearby.

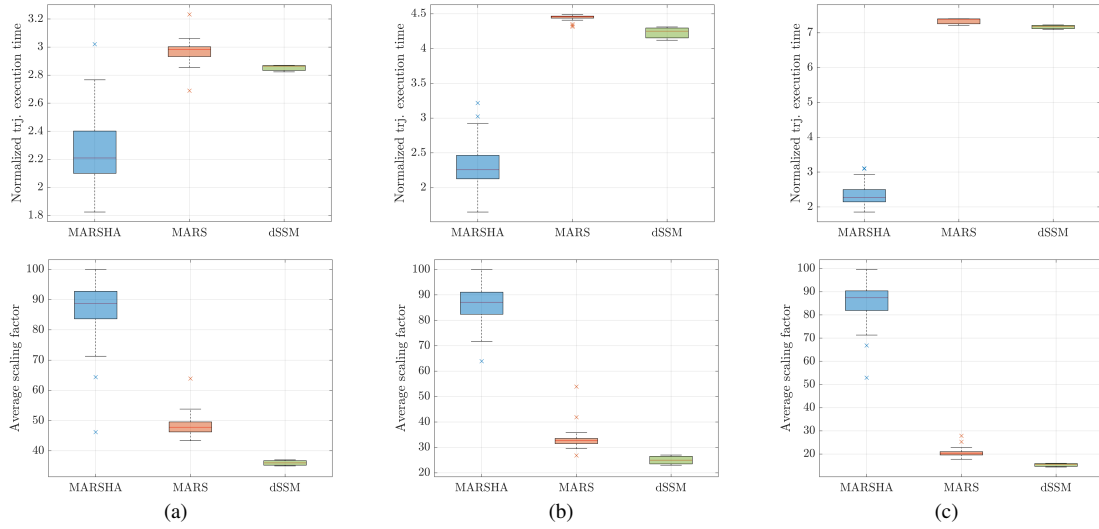


Fig. 5: Simulation comparison with reactive approaches. (a) *short* interaction (5 seconds); (b) *medium* interaction (10 seconds); (c) *long* interaction (20 seconds).

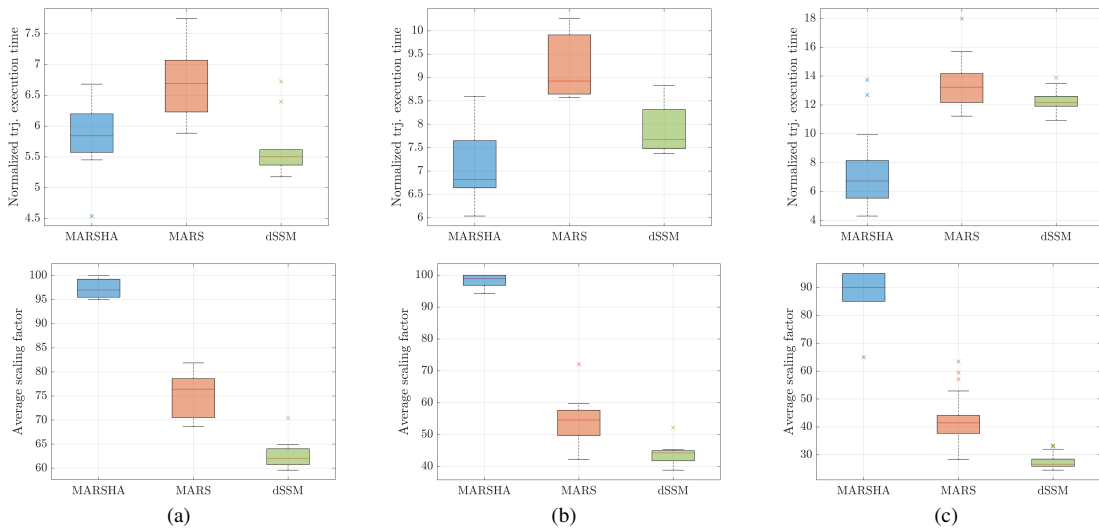


Fig. 6: Real experiments with reactive approaches. (a) *short* interaction (5 seconds); (b) *medium* interaction (10 seconds); (c) *long* interaction (20 seconds).

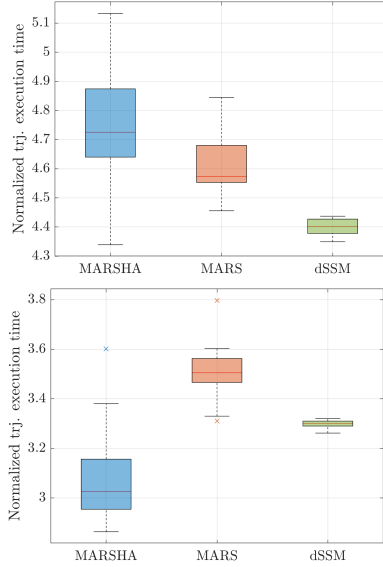


Fig. 7: Comparison with reactive approaches. Maximum robot speed scaled to 30% (top) and 60% (bottom).

execution. As the duration of the operator’s presence in the shared space increases, MARSHA performs better than MARS/dSSM. However, the performance disparity between the algorithms narrows. This can be attributed to the robot operating at 30% of its total capacity during real-world tests. Consequently, the distance the robot covers before the operator departs is reduced. As a result, the robot often does not reach the portion of the path replanned by MARSHA before the operator leaves the workspace. In this way, MARSHA’s advantages are not fully utilized, leading to a comparable performance with dSSM as the robot’s maximum speed decreases and the duration of the operator’s presence at the workstation diminishes. To confirm this hypothesis, we conducted simulated *short* tests with constant scaling at 30% and 60%, combined with the scaling provided by the safety module. As depicted in Fig. 7, reducing the robot’s speed corresponds to a smaller performance gap between the algorithms. When the robot’s speed is restricted, and the operator’s interventions are brief and infrequent, MARSHA performs as dSSM. Additionally, other factors, such as the positioning of the operator and the timing of entry into the cell, may influence the results, as they are not always the same during real-world tests.

E. Comparison with Proactive Approaches

1) *Simulation*: In the tests, the robot moves from a starting position to a goal while the operator works in the shared space for 20 seconds. When the initial path is calculated, the operator is already near the work table, allowing proactive approaches to factor in the operator’s position during the planning phase. Each algorithm (HAMP [17], MARSHA, and MIN-LEN [49]) was tested with 50 simulations and 20 trials in a real cell. HAMP

plans a path that minimizes execution time by considering the operator’s position during the planning phase, while MIN-LEN finds the shortest path while avoiding collisions with the person. MARSHA was tested with initial paths calculated using both HAMP and MIN-LEN. Comparing these two methodologies helps us understand if the initial path calculation heavily influences MARSHA’s performance. The results are illustrated in Fig. 8 (a) and (b). MARSHA_{LEN} refers to MARSHA with the initial path computed using the MIN-LEN method.

The conducted tests yield some considerations. When the initial path of MARSHA is safety-aware, HAMP and MARSHA exhibit comparable execution times. In some cases, MARSHA even outperforms HAMP by efficiently accommodating deviations caused by the operator. The fixed 20-second planning time of HAMP can lead to suboptimal solutions, whereas MARSHA’s ability to improve the plan during execution results in time savings. As expected, MIN-LEN performs poorly due to its path’s excessive proximity to the operator, which results in significant slowdowns of the shortest path. However, an intriguing result emerges when the initial path of MARSHA is generated using MIN-LEN. Even without human-awareness on the initial path, MARSHA’s performance remains relatively unaffected compared to using a human-aware path. MARSHA can provide performance similar to a human-aware planner using a simpler and faster generic initial path calculation that takes just a few seconds instead of tens of seconds. This outcome solidifies MARSHA’s position as a hybrid control strategy, skillfully combining elements from proactive and reactive planners and showcasing unique characteristics from both categories.

2) *Real scenario*: We also observe a narrowing gap between MIN-LEN and the other algorithms in real experiments. This phenomenon is related to the constant scaling at 30%. When the operator is near the table, the robot halts, rendering the 30% scaling effect insignificant during those 20 seconds of stoppage. However, when the operator is far, allowing the robot to move freely, it follows the shortest path, and the constant scaling affects a short trajectory. Conversely, the other algorithms maintain a certain distance from the operator, leading to longer travels at a reduced speed of 30%. Consequently, the impact of the constant scaling becomes more pronounced for these algorithms. As in Section VI-D2, we repeated the simulations under constant speed scaling conditions of both 30% and 60%, combined with the scaling imposed by the safety module. The results shown in Fig. 8 (c) demonstrate that lower robot velocities tend to reduce performance disparities between the algorithms.

MARSHA displays superior adaptability to HAMP when the operator’s behavior deviates from expectations, such as moving or walking away. HAMP’s trajectory experiences slowdowns when the operator moves unexpectedly within the workspace, as it lacks online adapta-

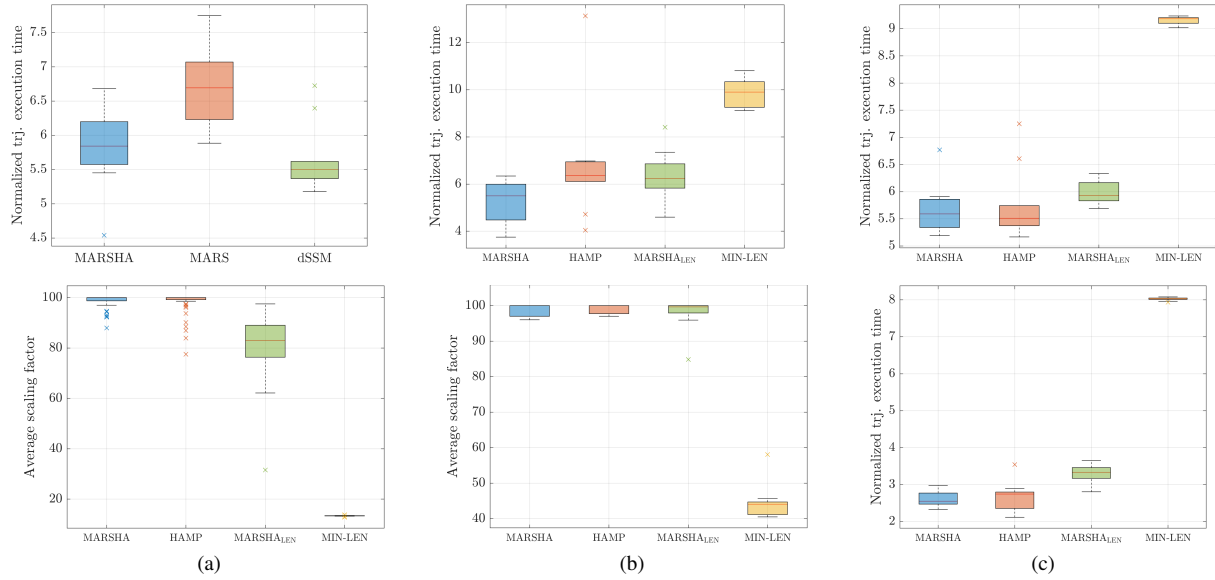


Fig. 8: Test with proactive approaches. (a) simulations; (b) real experiments; (c) maximum robot speed scaled to 30% (top) and 60% (bottom).

tion capabilities. Moreover, if the operator moves away from the designated area, HAMP continues to follow the same trajectory as if the operator were still in the shared workspace, resulting in a longer trajectory. In contrast, MARSHA quickly adjusts its plan, guiding the robot along the optimal trajectory to accommodate the operator's movements within the workspace or when the operator exits. These results are visually demonstrated in the accompanying video, available at [19].

F. Effect of the SSM Parameters on the Performance

This section analyzes the impact of SSM parameters (7) on MARSHA's performance and how the performance gap with MARS/dSSM changes accordingly.

Section VI-C outlines the parameters used in the tests described in Sections VI-D1 through VI-E2. By modifying these parameters, the degree to which the safety module intervenes to decelerate the robot varies. Certain parameter combinations prompt immediate and pronounced reactions from the safety module as soon as the operator enters the shared workspace, while other combinations allow for delayed intervention. The risk assessment must determine these parameters according to the system reaction times (T_r) and the robot's maximum Cartesian acceleration (a_s).

Two values were chosen for each parameter, resulting in 16 combinations. For each combination, the *long* test was repeated 50 times for each algorithm. Table II presents the selected parameter values, including those representing a more reactive cell and those simulating a less reactive cell. Two-speed values were considered for the human operator: 0 m/s and 1.6 m/s, which aligns with the recommended

value by ISO/TS 15066 when measuring the human walking speed is not feasible. Fig. 9 shows that varying sets of parameters have distinct effects on the performance of MARSHA. MARSHA outperforms MARS/dSSM in most cases and demonstrates comparable performance in the worst-case scenarios. With conservative implementations, MARSHA's behavior aligns with that of dSSM. The minimum allowed human-robot distance prevents MARSHA from generating solutions that do not trigger safety interventions. MARS achieves good performance levels when less conservative implementations allow for a smaller safety distance. In the remaining cases, MARSHA provides shorter execution times. Similar considerations can be drawn for PFL parameters.

G. Summary of Results and Discussion

Compared to reactive approaches (MARS and dSSM), MARSHA reduces the interventions of the safety module and reach the goal faster, especially during long operator interactions. The advantages are less significant for sporadic and brief interventions. This result highlights the importance of the application's features in selecting the most effective planning approach. Safety module configurations influence MARSHA's performance: conservative settings yield a behavior similar to dSSM, while with less conservative settings MARS already performs well. In the remaining scenarios, MARSHA achieves shorter execution times. MARSHA outperformed offline planners (HAMP and MIN-LEN) by improving real-time solutions and adjusting to unforeseen operator movements. Through online optimization, MARSHA outperforms even offline

TABLE II: Values used for each parameter of (7).

set	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
C (m)	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.30	0.30	0.30	0.30	0.30	0.30	0.30	0.30
T_r (s)	0.15	0.15	0.15	0.15	0.30	0.30	0.30	0.30	0.15	0.15	0.15	0.15	0.30	0.30	0.30	0.30
v_h (m/s)	0.00	0.00	1.60	1.60	0.00	0.00	1.60	1.60	0.00	0.00	1.60	1.60	0.00	0.00	1.60	1.60
a_s (m/s ²)	0.10	2.50	0.10	2.50	0.10	2.50	0.10	2.50	0.10	2.50	0.10	2.50	0.10	2.50	0.10	2.50

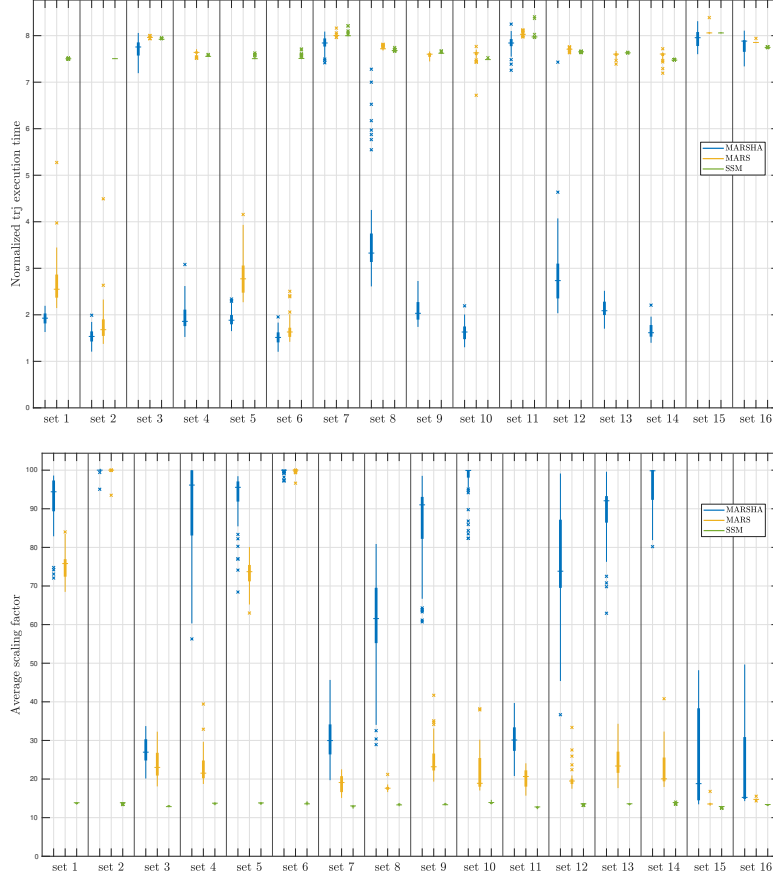


Fig. 9: Long test with the SSM parameter sets in Table II.

planners when its initial path is not human-aware. This confirms the advantages of its proactive-reactive nature.

VII. CONCLUSIONS

We presented MARSHA, a motion replanning algorithm to enhance human-robot cooperation. MARSHA dynamically estimates safety stops and slowdowns owing to human-robot proximity and continuously adjusts the robot's path in real time to minimize the estimated trajectory execution time. Unlike previous works, MARSHA blends reactivity with safety-aware proactivity. Compared to purely reactive or proactive approaches, MARSHA reduces the need for safety interventions, especially in long-term human-robot interactions. Its hybrid approach opens new possibilities for seamless human-robot cooperation across various application domains. One potential limitation is its reliance on robust perception systems, which can be challenging in unstructured environments where

obtaining reliable scene information may be difficult. Future work will extend the algorithm to explore additional interaction factors, such as legibility and ergonomics.

REFERENCES

- [1] S. Robla-Gómez, V. M. Becerra, J. R. Llata, E. González-Sarabia, C. Torre-Ferrero, and J. Pérez-Oria, "Working together: A review on safe human-robot collaboration in industrial environments," *IEEE Access*, vol. 5, pp. 26 754–26 773, 2017.
- [2] F. Vicentini, "Terminology in safety of collaborative robotics," *Robot. Comput.-Integr. Manuf.*, vol. 63, p. 101921, 2020.
- [3] International Organization for Standardization, "ISO/TS 15066:2016 Robots and robotic devices – Collaborative robots," Geneva, CH, Standard, Feb. 2016.
- [4] S. Sandrini, M. Faroni, and N. Pedrocchi, "Learning action duration and synergy in task planning for human-robot collaboration," in *2022 IEEE 27th ETFA*, 2022, pp. 1–6.
- [5] M. Faccio, I. Granata, and R. Minto, "Task allocation model for human-robot collaboration with variable cobot speed," *J of Intelligent Manufacturing*, pp. 1–14, 2023.
- [6] C.-M. Huang and B. Mutlu, "Anticipatory robot control for efficient human-robot collaboration," in *2016 11th ACM/IEEE Int Conf on Human-Robot Interaction (HRI)*, 2016, pp. 83–90.

- [7] S. Proia, R. Carli, G. Cavone, and M. Dotoli, "Control techniques for safe, ergonomic, and efficient human-robot collaboration in the digital industry: A survey," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 1798–1819, 2022.
- [8] A. Pupa, M. Minelli, and C. Secchi, "A Time-Optimal Energy Planner for Safe Human-Robot Collaboration," in *IEEE ICRA*, 2024, pp. 17 373–17 379.
- [9] A. Palleschi, M. Hamad, S. Abdolshah, M. Garabini, S. Haddadin, and L. Pallottino, "Fast and safe trajectory planning: Solving the cobot performance/safety trade-off in human-robot shared environments," *IEEE Robot. Autom. Lett.*, vol. 6, pp. 5445–5452, 2021.
- [10] N. Lucci, B. Lacevic, A. M. Zanchettin, and P. Rocco, "Combining speed and separation monitoring with power and force limiting for safe collaborative robotics applications," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6121–6128, 2020.
- [11] C. Byner, B. Matthias, and H. Ding, "Dynamic speed and separation monitoring for collaborative robot applications - concepts and performance," *Robot. and CIM*, vol. 58, pp. 239–252, 2019.
- [12] J. A. Marvel and R. Norcross, "Implementing speed and separation monitoring in collaborative robot workcells," *Robot. Comput. Integr. Manuf.*, vol. 44, pp. 144–155, 2017.
- [13] C. Tonola, M. Faroni, N. Pedrocchi, and M. Beschi, "Anytime informed path re-planning and optimization for human-robot collaboration," in *2021 30th IEEE Int Conf on Robot & Human Interactive Communication (RO-MAN)*, 2021, pp. 997–1002.
- [14] C. Tonola, M. Faroni, M. Beschi, and N. Pedrocchi, "Anytime informed multi-path replanning strategy for complex environments," *IEEE Access*, vol. 11, pp. 4105–4116, 2023.
- [15] M. Hüppi, L. Bartolomei, R. Mascaro, and M. Chli, "T-PRM: Temporal Probabilistic Roadmap for Path Planning in Dynamic Environments," in *IEEE/RSJ IROS*, 2022, pp. 10 320–10 327.
- [16] A. Casalino, D. Bazzi, A. M. Zanchettin, and P. Rocco, "Optimal proactive path planning for collaborative robots in industrial contexts," in *IEEE ICRA*, 2019, pp. 6540–6546.
- [17] M. Faroni, M. Beschi, and N. Pedrocchi, "Safety-aware time-optimal motion planning with uncertain human state estimation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 12 219–12 226, 2022.
- [18] J. Mainprice, E. Akin Sisbot, L. Jaillet, J. Cortés, R. Alami, and T. Siméon, "Planning human-aware motions using a sampling-based costmap planner," in *IEEE ICRA*, 2011, pp. 5012–5017.
- [19] C. Tonola, "Video - Reactive and safety-aware path replanning for collaborative applications." [Online]. Available: <https://youtu.be/r4rLiNMFRXk>
- [20] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.
- [21] C. Tonola, "marsha_examples" [Online]. Available: https://github.com/JRL-CARI-CNR-UNIBS/marsha_examples
- [22] E. A. Sisbot and R. Alami, "A human-aware manipulation planner," *IEEE Tran on Robotics*, vol. 28, no. 5, pp. 1045–1057, 2012.
- [23] R. Hayne, R. Luo, and D. Berenson, "Considering avoidance and consistency in motion planning for human-robot manipulation in a shared workspace," in *2016 IEEE ICRA*, 2016, pp. 3948–3954.
- [24] S. Tarbouriech and W. Suleiman, "Bi-objective motion planning approach for safe motions: Application to a collaborative robot," *J. of Intell. & Rob. Sys.*, vol. 99, pp. 45–63, 2020.
- [25] X. Zhao and J. Pan, "Considering human behavior in motion planning for smooth human-robot collaboration in close proximity," in *IEEE ROMAN*, 2018, pp. 985–990.
- [26] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *Proc. IEEE ICRA*, Shanghai (China), 2011, pp. 4569–4574.
- [27] K. Cai, W. Chen, C. Wang, S. Song, and M. Q.-H. Meng, "Human-Aware Path Planning With Improved Virtual Doppler Method in Highly Dynamic Environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1304–1321, 2023.
- [28] A. Kanazawa, J. Kinugawa, and K. Kosuge, "Adaptive motion planning for a collaborative robot based on prediction uncertainty to enhance human safety and work efficiency," *IEEE Trans. Robot.*, vol. 35, no. 4, pp. 817–832, 2019.
- [29] P. A. Lasota and J. A. Shah, "Analyzing the effects of human-aware motion planning on close-proximity human-robot collaboration," *Human Factors*, vol. 57, no. 1, pp. 21–33, 2015.
- [30] M. Beschi, M. Faroni, C. Copot, and N. Pedrocchi, "How motion planning affects human factors in human-robot collaboration," *IFAC-PapersOnLine*, vol. 53, no. 5, pp. 744–749, 2020.
- [31] R. Laha, W. Wu, R. Sun, N. Mansfeld, L. F. Figueredo, and S. Haddadin, "S*: On safe and time efficient robot motion planning," in *2023 IEEE ICRA*, 2023, pp. 12 758–12 764.
- [32] J. Flowers, M. Faroni, G. Wiens, and N. Pedrocchi, "Spatio-temporal avoidance of predicted occupancy in human-robot collaboration," in *2023 32nd IEEE Int Conf on Robot and Human Interactive Communication (RO-MAN)*, 2023, pp. 2162–2168.
- [33] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong Planning A*," *Artificial Intelligence*, vol. 155, no. 1, pp. 93–146, 2004.
- [34] S. Koenig and M. Likhachev, "Fast replanning for navigation in unknown terrain," *IEEE T-RO*, vol. 21, no. 3, pp. 354–363, 2005.
- [35] Y. Hwang and N. Ahuja, "Path planning using a potential field representation," in *CVPR*, 1989, pp. 569–575.
- [36] G. Chirriatti, G. Palmieri, C. Scoccia, M. C. Palpacelli, and M. Callegari, "Adaptive obstacle avoidance for a class of collaborative robots," *Machines*, vol. 9, no. 6, 2021.
- [37] H. Liu, D. Qu, F. Xu, Z. Du, K. Jia, J. Song, and M. Liu, "Real-time and efficient collision avoidance planning approach for safe human-robot interaction," *J. of Intell. & Rob. Sys.*, 2022.
- [38] G. Nicola and S. Ghidoni, "Deep reinforcement learning for motion planning in human robot cooperative scenarios," in *2021 26th IEEE ETFA*, 2021, pp. 01–07.
- [39] S. Luo and L. Schomaker, "Reinforcement learning in robotic motion planning by combined experience-based planning and self-imitation learning," *Rob and Aut Sys*, vol. 170, p. 104545, 2023.
- [40] P. Chen, J. Pei, W. Lu, and M. Li, "A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance," *Neurocomputing*, vol. 497, pp. 64–75, 2022.
- [41] W. Liu, K. Eltoumy, S. Tian, X. Liang, and M. Zheng, "KG-Planner: Knowledge-Informed Graph Neural Planning for Collaborative Manipulators," *IEEE T-ASE*, pp. 1–11, 2024.
- [42] D. Ferguson, N. Kalra, and A. Stentz, "Replanning with RRTs," in *Proc. IEEE Int Conf on Rob and Aut*, 2006, pp. 1243–1248.
- [43] Z. Shen, J. P. Wilson, S. Gupta, and R. Harvey, "SMART: Self-Morphing Adaptive Replanning Tree," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7312–7319, 2023.
- [44] M. Yavari, K. Gupta, and M. Mehrandezh, "Interleaved predictive control and planning for an unmanned aerial manipulator with on-the-fly rapid re-planning in unknown environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 3, pp. 1690–1705, 2023.
- [45] H. Pham and Q.-C. Pham, "A new approach to time-optimal path parameterization based on reachability analysis," *IEEE Tran on Robotics*, vol. 34, no. 3, pp. 645–659, 2018.
- [46] S. Haddadin, S. Haddadin, A. Khoury, T. Rokahr, S. Parusel, R. Burgkart, A. Bicchi, and A. Albu-Schäffer, "On making robots understand safety: Embedding injury knowledge into control," *Int. J. Robot. Res.*, vol. 31, no. 13, pp. 1578–1602, 2012.
- [47] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *IEEE/RSJ Int Conf on Intelligent Robots and Systems*, 2014, pp. 2997–3004.
- [48] S. LaValle, "Rapidly-exploring random trees: a new tool for path planning," *The annual research report*, 1998.
- [49] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *IJRR*, vol. 30, no. 7, pp. 846–894, 2011.
- [50] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Batch informed trees (BIT*): Informed asymptotically optimal anytime search," *Int. J. Robot. Res.*, vol. 39, no. 5, pp. 543–567, 2020.
- [51] J. Wang, W. Chi, C. Li, C. Wang, and M. Q.-H. Meng, "Neural RRT*: Learning-Based Optimal Path Planning," *IEEE Tran on Aut Sci and Eng*, vol. 17, no. 4, pp. 1748–1758, 2020.
- [52] J. Krüger, T. Lien, and A. Verl, "Cooperation of human and machines in assembly lines," *CIRP Annals*, vol. 58, no. 2, pp. 628–646, 2009.
- [53] M. Faroni, N. Pedrocchi, and M. Beschi, "Adaptive hybrid local-global sampling for fast informed sampling-based optimal path planning," *Autonomous Robots*, vol. 48, no. 2–3, Apr. 2024.
- [54] C. Tonola, M. Beschi, M. Faroni, and N. Pedrocchi, "OpenMORE: an open-source tool for sampling-based path replanning in ROS," in *2023 IEEE 28th ETFA*, 2023, pp. 1–4.